

# Technical Working Blueprint

*A beginner-friendly build plan showing pages, files, data and pair-to-pair handoffs*

**PURPOSE** Let all three pairs begin together using the same structure and the same sample answers.

ONE TEAM • THREE PAIRS • ONE CONNECTED PROTOTYPE

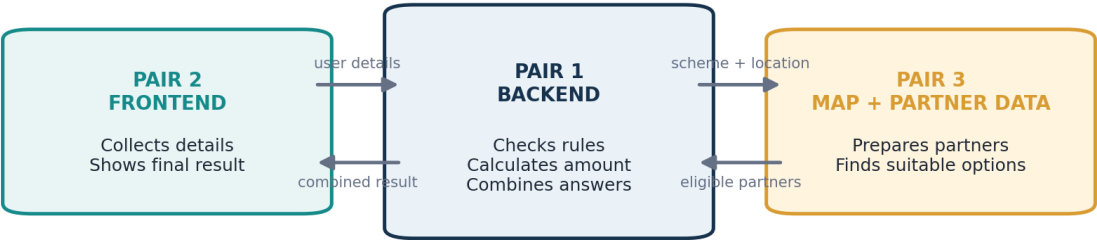
|                                                           |                                                                  |                                                                          |
|-----------------------------------------------------------|------------------------------------------------------------------|--------------------------------------------------------------------------|
| <p><b>PAIR 1</b><br/>Backend</p> <p>(Ruchit and Jeet)</p> | <p><b>PAIR 2</b><br/>Frontend</p> <p>(Rudra and Bhagyashree)</p> | <p><b>PAIR 3</b><br/>Map and Partner Data</p> <p>(Jaymin and Anjali)</p> |
|-----------------------------------------------------------|------------------------------------------------------------------|--------------------------------------------------------------------------|

*10-page working guide | Read together before implementation*

# 01 | The simple technical picture

Technical words are used only when they help someone build a real part

## How the three pairs connect



Frontend sends details; backend combines rules and partner results; frontend displays the answer

| Part       | Chosen technology             | Plain meaning                                                                  |
|------------|-------------------------------|--------------------------------------------------------------------------------|
| Frontend   | React + Vite + Bootstrap      | The pages and buttons the user sees                                            |
| Backend    | Python + FastAPI              | The hidden logic that checks rules and returns answers                         |
| Data       | JSON files                    | Simple files holding schemes and partners; no database for the first prototype |
| Map        | React Leaflet + OpenStreetMap | Displays markers and the user's nearby suitable partners                       |
| Connection | HTTP API using JSON           | A fixed message format used to exchange information between parts              |

**FROZEN CHOICE** Do not allow different coding agents to replace this stack. A simpler shared stack is more valuable than six different advanced suggestions.

## 02 | The project structure

Every file has one owner so coding agents do not overwrite another pair's work

### Recommended folders

```
sih26092/
├── frontend/src/
│   ├── pages/                # Pair 2: Home, Profile, Result, Partners, Summary
│   ├── components/
│   │   └── PartnerMap.jsx    # Pair 3: reusable map component
│   ├── mock/sampleData.js    # Pair 2: temporary sample answers
│   ├── services/api.js       # Pair 2: calls backend
│   └── App.jsx               # Pair 2: page navigation
├── backend/
│   ├── app.py                # Pair 1: FastAPI starting point
│   ├── routes/               # Pair 1: recommend, calculate, partners
│   ├── services/
│   │   ├── recommender.py    # Pair 1
│   │   ├── calculator.py     # Pair 1
│   │   └── partner_locator.py # Pair 3 module, connected by Pair 1
│   ├── data/
│   │   ├── schemes.json      # Pair 1
│   │   └── partners.json     # Pair 3
│   └── tests/                # Each pair tests its own module
└── README.md                 # Shared run instructions
```

| Pair   | Files it controls                                 | Rule                                                            |
|--------|---------------------------------------------------|-----------------------------------------------------------------|
| Pair 1 | backend app, routes, scheme rules, calculation    | May import Pair 3's locator; must not rewrite map/data files    |
| Pair 2 | pages, navigation, sample responses, API caller   | Uses fixed response fields; must not change backend names alone |
| Pair 3 | partners.json, partner_locator.py, PartnerMap.jsx | Gives tested modules for Pair 1 and Pair 2 to import            |

**ONE OWNER PER FILE** Both people inside a pair may work together, but only one person should edit a shared file at a time.

### 03 | The shared information contract

*The contract lets all three pairs work before the real connection is ready*

| User field           | Example          | Why it is needed                                |
|----------------------|------------------|-------------------------------------------------|
| has_sc_certificate   | true             | Basic target-group check                        |
| annual_income        | 300000           | Checks the configured income limit              |
| purpose              | business         | Separates business and education journeys       |
| project_cost         | 380000           | Helps select the scheme and calculate the loan  |
| activity_type        | dairy            | Adds understandable context                     |
| state / district     | Gujarat / Rajkot | Helps find suitable partners                    |
| latitude / longitude | optional         | Helps sort by distance when location is allowed |

Sample request sent by Pair 2 to Pair 1

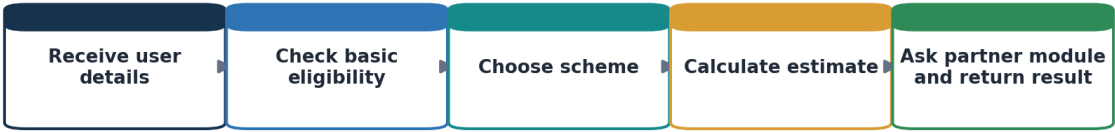
```
{
  "has_sc_certificate": true,
  "annual_income": 300000,
  "purpose": "business",
  "project_cost": 380000,
  "activity_type": "dairy",
  "state": "Gujarat",
  "district": "Rajkot"
}
```

**FREEZE THE NAMES** Pair 2 may design with this sample request. Pair 1 must accept these same fields. A field changes only after all affected pairs agree.

## 04 | Pair 1 - Backend work

Turn user details into one clean answer for the frontend

### Pair 1 backend flow



The backend sequence

| Person A example                                                                      | Person B example                                                            |
|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Prepare schemes.json; write eligibility rules; create valid and invalid test profiles | Create FastAPI routes; build calculator; combine scheme and partner results |

| File/API                | What it must do                                                   |
|-------------------------|-------------------------------------------------------------------|
| data/schemes.json       | Store limits, rates, loan caps, tenure and moratorium values      |
| services/recommender.py | Return suitable scheme, alternative, reasons and rejected reasons |
| services/calculator.py  | Return possible loan, own contribution and estimated repayment    |
| POST /api/recommend     | Receive the user profile and return the scheme result             |
| POST /api/calculate     | Receive scheme/amount information and return the money estimate   |
| GET /api/partners       | Call Pair 3's locator and return suitable partners                |

**BACKEND DONE MEANS** The same sample input always returns the same valid JSON response, and boundary tests do not crash.

## 05 | Pair 1 - Rules and calculations

*Keep money decisions fixed, visible and testable*

| Scheme                  | Project/loan guide                                      | Rate guide | Simple prototype use                        |
|-------------------------|---------------------------------------------------------|------------|---------------------------------------------|
| Micro Finance           | Project up to ₹1.40 lakh; loan cap ₹1.25 lakh           | 6.5%       | Small business activity                     |
| Aajeevika Micro-Finance | Project up to ₹1.40 lakh; loan cap ₹1.25 lakh           | 15%        | Selected NBFC-MFI route                     |
| Term Loan               | Project above ₹1.40 lakh to ₹50 lakh; loan cap ₹45 lakh | 8%         | Larger business project                     |
| Udyam Nidhi             | Project up to ₹5 lakh; loan cap ₹4.50 lakh              | 13% / 15%  | Small/micro alternative                     |
| Educational Loan        | Up to ₹40 lakh or 90% of course fee                     | 6.5%       | Recognized professional/technical education |

**COMMON ELIGIBILITY** For this prototype, the applicant has an SC certificate and annual family income does not exceed ₹5 lakh.

### Calculation guide

```
possible_loan = min(project_cost * 0.90, scheme_loan_cap)
own_contribution = project_cost - possible_loan

monthly_rate = annual_rate / 12 / 100
monthly_estimate = P * r * (1+r)^n / ((1+r)^n - 1)
```

- Show the calculation as an estimate and display the assumption used for the moratorium.
- Never use an LLM to calculate eligibility or money values.
- Test exactly at and just above ₹5 lakh income and ₹1.40 lakh project cost.

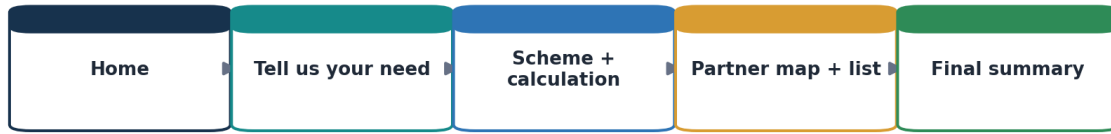
**SOURCE RULE** Copy official values into schemes.json with source\_url and checked\_date. Change the JSON, not many frontend files, when a rule changes.

## 06 | Pair 2 - Frontend work

*Build the full journey immediately using sample responses*

### Five simple screens

One screen leads naturally to the next



*The five frontend pages*

| Page     | Main content                                                 | Data expected                         |
|----------|--------------------------------------------------------------|---------------------------------------|
| Home     | Project explanation, language, Start                         | No backend data                       |
| Profile  | SC certificate, income, purpose, cost, activity, location    | Creates the shared request            |
| Result   | Recommended scheme, reasons, alternative, loan and repayment | Recommendation + calculation response |
| Partners | Map, ranked cards, address and reason                        | Partner search response               |
| Summary  | Profile + selected result in one printable view              | Stored result from earlier pages      |

| Person A example                                  | Person B example                                     |
|---------------------------------------------------|------------------------------------------------------|
| Build Home, Profile, validation and shared layout | Build Result, Partners, Summary and response display |

**HOW PAIR 2 STARTS NOW** Store the agreed sample JSON in mock/sampleData.js. Design every result screen from that sample, then replace the sample with fetch() calls when Pair 1 is ready.

- Show loading, no-result and simple error messages.
- Use plain labels; do not display internal JSON field names to the user.
- Do not add charts unless the five-page journey already works.

## 07 | Pair 3 - Map and Partner Data from zero

One pair prepares trustworthy partner records and turns them into a useful nearby-partner view

### What the map work actually means



Map work explained as five small tasks

| Person A: Partner Data                                                                                                                 | Person B: Map Function                                                                                             |
|----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Collect 10-15 official demo partners; clean names and addresses; add source, service area, schemes and clearly marked prototype status | Save coordinates; get user location or district; filter and sort results; build matching markers and partner cards |

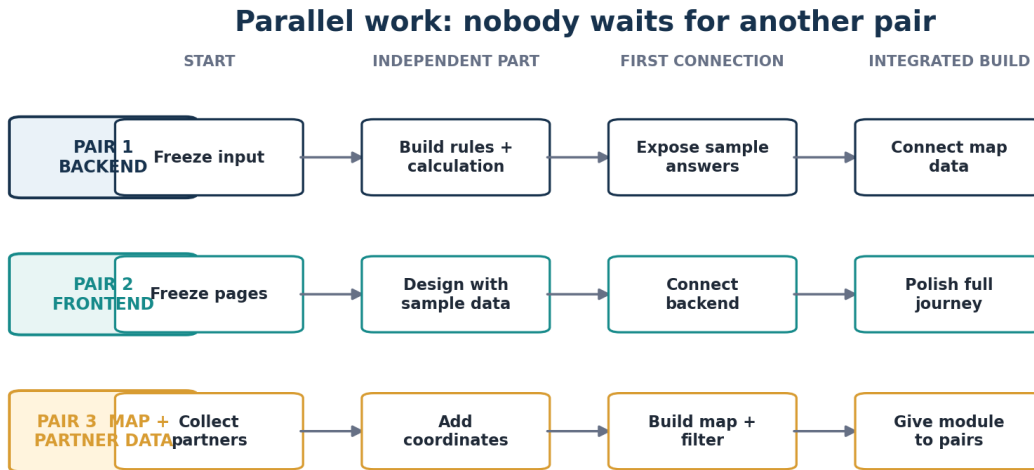
| File               | What it contains or does                                                                                      | Who receives it |
|--------------------|---------------------------------------------------------------------------------------------------------------|-----------------|
| partners.json      | ID, name, type, state, district, address, latitude, longitude, supported schemes, source and prototype status | Pair 1          |
| partner_locator.py | Filters by service area, scheme and demo eligibility; then sorts by distance when coordinates exist           | Pair 1          |
| PartnerMap.jsx     | Shows user position, partner markers and the same partner IDs used by the result cards                        | Pair 2          |

| Filtering order | Check                                                       |
|-----------------|-------------------------------------------------------------|
| 1               | Partner serves the user's state/service area                |
| 2               | Partner supports the recommended scheme                     |
| 3               | Prototype 'fund available' and 'no overdues' flags are true |
| 4               | Sort by distance when coordinates are available             |
| 5               | Return top results with a plain-language reason             |

**HANDOFF + DATA HONESTY** Turn each address into map coordinates once and save them. Give partners.json and partner\_locator.py to Pair 1; give PartnerMap.jsx and one sample response to Pair 2. Keep fund, overdue and scheme-support fields labelled as prototype data unless an official live source is connected.

## 08 | All three pairs working in parallel

Use frozen examples first; replace them with real connections at the checkpoint



Parallel work and handoff points

| Handoff         | What is passed                                  | Acceptance check                           |
|-----------------|-------------------------------------------------|--------------------------------------------|
| Pair 2 → Pair 1 | Final input fields and page data needs          | Backend accepts the same field names       |
| Pair 1 → Pair 2 | Sample and real recommendation/calculation JSON | Result page renders without manual edits   |
| Pair 3 → Pair 1 | Partner data and locator service                | Partners endpoint returns filtered results |
| Pair 3 → Pair 2 | Map component and partner response example      | Markers/cards display the same partner IDs |

**CHANGE RULE** No coding agent may rename an endpoint, file or response field without a written change accepted by every affected pair.

### Minimum integration test

1. Open Home and submit the Rajkot dairy profile.
2. See Term Loan, reasons, amount estimate and alternative.
3. Open Partners and see filtered map markers/cards.
4. Select one partner and open the final Summary.

## 09 | Build checks, fallbacks and completion

*Protect the demonstration before adding extra features*

| If this fails         | Use this safe fallback                                               |
|-----------------------|----------------------------------------------------------------------|
| Backend unavailable   | Pair 2 switches to the same sample JSON so the UI can still be shown |
| Map tiles unavailable | Show ranked partner cards with address and reason                    |
| User blocks location  | Use selected state/district instead of live coordinates              |
| Deployment fails      | Run frontend and backend locally and keep a recorded demo            |
| Language incomplete   | Show complete English/Hindi rather than a broken third language      |

### Definition of done

- The owned files are committed and another pair can run them from the README.
- One valid and one invalid/boundary case are tested.
- The result uses the fixed field names and contains no blank screen or unhandled error.
- Prototype partner-status data is visibly labelled.
- The five-page user journey works before optional features begin.

**FINAL PRIORITY** Working input → recommendation → calculation → partners → summary is more valuable than any unfinished chart, chatbot or dashboard.

### Official references used

- SIH 2026 problem statements: [Open official page](#)
- NSFDCL loan and credit schemes: [Open official page](#)
- NSFDCL Channel Partner lists: [Open official page](#)
- NSFDCL eligibility and channel-finance FAQs: [Open official page](#)